

Relatório de Auditoria de Segurança — ERP-VSD

Data: 31 de agosto de 2026

Escopo

Monorepo Maven multi-módulo em Java 25 / Spring Boot 4.x / Spring Cloud 2025.x (registry, gateway, auth-service, cadastro-service, partner-service, billing-service, fiscal-service, liquibase-service) + frontend Angular 21 standalone em 3 SPAs (erp-front-end-web, erp-front-end-admin, erp-front-end-partner). ORM: Spring Data JPA (Hibernate) com Liquibase controlando todo DDL. Auth: JWT HS256 emitido pelo auth-service, validado no gateway (SecurityFilter), que injeta X-Tenant-Id/X-User-Id/X-Is-Owner/authorities como headers para os serviços downstream; downstream services não revalidam a assinatura do JWT, apenas confiam nos headers do gateway, e usam @PreAuthorize (RBAC método-a-método) para autorização granular. Deploy: Docker (Dockerfile por serviço) + docker-compose (infra local) + Jenkins/SonarQube (CI/CD declarativo via Jenkinsfile).

Metodologia

Cada categoria genérica foi mapeada para o equivalente desta stack: (1) BANCO SEM TRANCA -> filtro de tenant via Hibernate @Filter/@FilterDef + AspectJ (TenantFilterAspect) + ThreadLocal (TenantContext), cuja limitação documentada é não cobrir acesso por chave primária (findById/deleteById) — verificado call-a-call; (2) PERMISSÃO NO NAVEGADOR -> cruzamento de *ngIf/@if e route guards Angular com anotações @PreAuthorize/SecurityConfig nos controllers Spring correspondentes; (3) IDOR -> varredura de todo handler REST que recebe um ID (path/query/body) checando ownership/tenant antes de ler, alterar ou excluir; (4) CHAVES EXPOSTAS -> grep de padrões \${VAR:-default} em application*.yaml/properties, compose.yaml, Dockerfiles, Jenkinsfile e histórico git recente; (5) INPUTS SEM TRATAMENTO -> grep de innerHTML/bypassSecurityTrust/eval no Angular e de concatenação de dado de usuário em HTML/e-mail no backend.

Categorias avaliadas

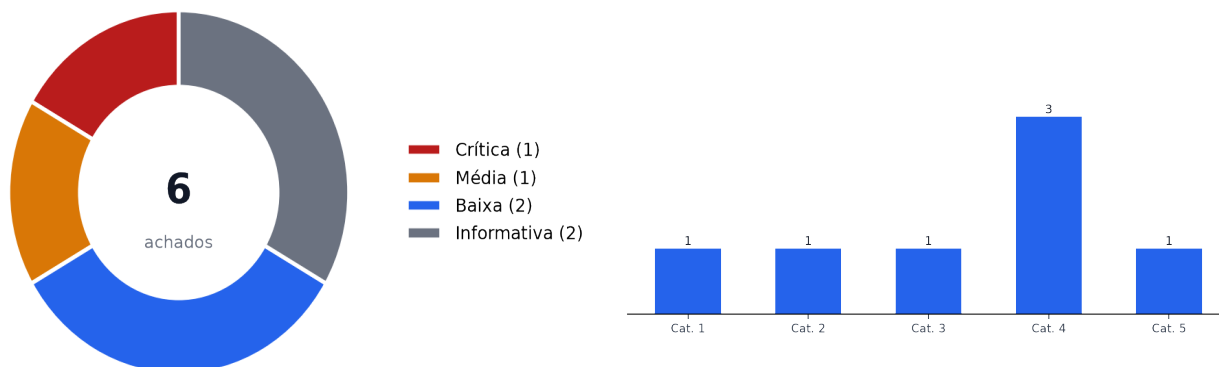
#	Categoria
1	Banco sem tranca (isolamento de tenant)
2	Permissão definida no navegador
3	IDOR
4	Chaves expostas
5	Inputs sem tratamento (XSS/HTML injection)

Resumo executivo

6 achados verificados em código real (1 crítico, 0 alto, 1 médio, 2 baixo, 2 informativo) e 8 pontos fortes confirmados com evidência, cobrindo os 5 serviços de aplicação (auth, cadastro, partner, billing, fiscal), o gateway e os 3 frontends Angular.

Achados por severidade

Achados por categoria



Principal risco

F1 — partner-service: PartnerController sem nenhuma autorização — CRUD administrativo aberto a qualquer usuário autenticado

partner-service não impõe autorização na cadeia de filtros do Spring Security (`anyRequest().permitAll()`), delegando 100% da autorização a `@PreAuthorize` por método — e `PartnerController` não tem NENHUM `@PreAuthorize`. O único gate que resta é o `InternalRequestFilter`, que apenas confirma que a requisição chegou via gateway com o segredo interno e um `X-User-Id` presente — não valida `role/authority`. O gateway, por sua vez, só valida a assinatura do JWT e injeta os headers; não há restrição de role por rota para `/partner/**`. Curiosamente o gateway TEM um mecanismo pronto para esse tipo de restrição — `verifyPartnerURL()` em `SecurityFilter.java` (~linha 126) já restringe JWTs com `loginType=="PARTNER"` a um allowlist de prefixos (`/partner/api/v1/partners/me, /cnpj/, /billing/partner/`) — mas essa restrição não existe para nenhum OUTRO `loginType` (ex.: usuário comum de um tenant) alcançando as mesmas rotas administrativas. No frontend admin, a rota `/parceiros/contadores` é protegida só pelo `authGuard` (checa apenas se existe um token em `sessionStorage`, não o papel do usuário). Resultado: qualquer usuário autenticado no admin (ou qualquer portador de um JWT válido de `loginType` diferente de `PARTNER`) pode listar todos os parceiros da plataforma, ver/editar qualquer parceiro por ID, ver dados de engajamento/indicação de qualquer tenant por ID, e aprovar/reprovar/inativar qualquer parceiro.

Pontos fortes (verificados)

O que foi conferido em código real e está correto — evidência de cobertura da auditoria, não apenas ausência de achados.

● **cadastro-service: isolamento de tenant end-to-end consistente**

TenantInterceptor (OncePerRequestFilter) popula TenantContext (ThreadLocal) a partir do header X-Tenant-Id em toda requisição; TenantFilterAspect (@Before via AspectJ) ativa o filtro Hibernate @Filter/@FilterDef definido em BaseTenantEntity (tenant_id = :tenantId) em toda chamada de service/repository, cobrindo listagens/buscas/agregações. Os 4 pontos de acesso por chave primária (findById cru, não coberto pelo @Filter — limitação documentada no próprio Javadoc do aspecto) em CondicaoPagamentoService.java:127, DepositoService.java:109, GrupoClienteService.java:125 e PessoaService.java:89 têm todos guarda manual explícita logo em seguida (if (!entity.getTenantId().equals(tenantId)) throw BusinessException(TENANT_ASSOC_ERROR, BAD_REQUEST)). O único @DeleteMapping do serviço (ClienteController) usa deleteByIdAndTenantId (tenant já embutido na query).

● **InternalRequestFilter: gateway como único ponto de confiança para headers de tenant**

SecurityUtils.getCurrentTenantId() em cadastro-service lê o header X-Tenant-Id diretamente — isso só é seguro porque InternalRequestFilter (presente em cadastro-service e partner-service) valida, em toda rota não-pública, um segredo compartilhado (internal.gateway.secret) via comparação de tempo constante (MessageDigest.isEqual) mais a presença do header X-User-Id, garantindo que só o gateway (ou quem souber o segredo) alcança o serviço diretamente.

● **billing-service: SubscriptionController — toda operação cross-tenant é gated por authority**

listar (com filtro opcional de tenantId), mrr, reprocessar, cobranças e cancelarAdmin — todos anotados @PreAuthorize("hasAuthority('ASSINATURA_MANAGE')"). O endpoint self-service (POST /me/cancel) deriva o tenant do header X-Tenant-Id, nunca de um ID no path/body — nenhuma superfície de IDOR nesse controller. Os findById internos de SubscriptionService (linhas 162, 177, 185, 198) só são alcançados a partir desses métodos admin-gated.

● **auth-service: RBAC consistente em**

UserController/AttributionsController/RoleController/PermissionController

Todo endpoint administrativo cross-tenant (createUser, updateUserById, updateUserStatusById, unlockUserById, getUserById, searchUsers, getUserRoles, assignRolesToUser, removeRoleFromUser em UserController/AttributionsController; createRole/updateRole/deleteRole/assignPermissions em RoleController; CRUD de PermissionController) está anotado @PreAuthorize("hasRole('APP_OWNER') and hasAuthority('...')"). Os wrappers usados pelo portal do tenant (updateUserForTenant/updateUserStatusForTenant/unlockUserForTenant em UserService) chamam assertUserInTenant (findByIdAndTenantId) antes de qualquer operação — IDOR-safe por construção, confirmado também pelos testes em TenantSecurityControllerTest.java (assignRolesRejectsIdMismatch, assignPermissionsRejectsIdMismatch).

● **Frontends Angular: nenhum uso de innerHTML/bypassSecurityTrust/eval em 318 arquivos**

Varredura completa dos 3 apps (85 .html + 233 .ts) não encontrou nenhuma ocorrência de [innerHTML], DomSanitizer/bypassSecurityTrust*, SafeHtml/SafeUrl/SafeResourceUrl, Renderer2.setProperty(...,'innerHTML',...), eval(ou new Function(, e nenhuma lib de markdown/DOMPurify instalada. Toda interpolação de dado dinâmico usa {{ }} (escapa por padrão) ou property binding de atributo ([href]/[src]) apontando para URLs de resposta de API (Asaas: fatura, boleto, PIX), sanitizadas automaticamente pelo Angular (SecurityContext.URL/RESOURCE_URL).

● **Segredos reais sempre via variável de ambiente, sem fallback hardcoded**

JWT_SECRET, DB_USER/DB_PASS, KAFKA_BROKERS, EMAIL_USER/EMAIL_PASSWORD, ASAAS_API_KEY/ASAAS_WEBHOOK_TOKEN e INTERNAL_GATEWAY_SECRET usam apenas \${VAR} (sem default) em todos os 7 serviços com application.yml/.properties. auth-service valida tamanho mínimo do JWT_SECRET (32 chars) via @PostConstruct em TokenService.java. Nenhum Dockerfile (dos 7) embute ARG/ENV com valor de segredo. git log dos últimos 20 dias não traz nenhum segredo real commitado, apenas constantes de teste (test-secret-*, test-internal-secret) usadas só em *Test.java.

● **fiscal-service: categoria 1 (isolamento de tenant) não aplicável por design**

/fiscal/calculador é determinístico e sem persistência (não há tabela por tenant a isolar). A única superfície tenant-sensível é o feature flag de split payment (fiscal.split-payment.tenants, um allowlist de tenants elegíveis lido de config, não de dado de outro tenant), default desligado — condizente com o registrado em memória de sessões anteriores.

● **Frontends: nenhum gate de UI condicionado a papel/permissão em lugar nenhum**

Varredura sistemática de isAdmin/isOwner/canEdit/hasRole/hasPermission/hasAuthority e de todos os *ngIf/@if (250+ ocorrências) nos 3 apps não encontrou nenhum gate de UI baseado em papel — toda condicional é estado de domínio/formulário. Os tipos de resposta de login não armazenam roles/authorities/isOwner. Isso confirma que a arquitetura de autorização é deliberadamente backend-only (o front mostra os mesmos menus a qualquer usuário autenticado e confia 100% no @PreAuthorize do Spring Security) — um modelo válido, mas que exige que TODO endpoint sensível tenha sua própria guarda no backend, o que é justamente o que falha em F1 (partner-service).

Achados detalhados

Crítica

F1 — partner-service: PartnerController sem nenhuma autorização — CRUD administrativo aberto a qualquer usuário autenticado

Categoria: Permissão definida no navegador, IDOR

partner-service/src/main/java/com/erp/partnerservice/api/controllers/PartnerController.java	1-247
partner-service/src/main/java/com/erp/partnerservice/infra/config/SecurityConfig.java	15, 23-24
gateway/src/main/resources/application.yml	13-18
gateway/src/main/java/com/erp/gateway/security/SecurityFilter.java	~126 (verifyPartnerURL)
Angular/erp-front-end-admin/src/app/pages/cadastros/parceiros/parceiros.service.ts	1-53
Angular/erp-front-end-admin/src/app/util/authguard.ts	4-14

```
// SecurityConfig.java
.authorizeHttpRequests(auth -> auth.anyRequest().permitAll())

// PartnerController.java – nenhum método tem @PreAuthorize
@GetMapping
public ResponseEntity<...> findAll(...) { ... }
@GetMapping("/{id}")
public ResponseEntity<...> findById(@PathVariable UUID id) { ... }
@PatchMapping("/{id}/approve")
public ResponseEntity<...> approve(@PathVariable UUID id, ...) { ... }
@GetMapping("/referrals/by-tenant/{tenantId}")
public ResponseEntity<...> origemPorTenant(@PathVariable Long tenantId) { ... }
```

Por que é explorável: partner-service não impõe autorização na cadeia de filtros do Spring Security (`anyRequest().permitAll()`), delegando 100% da autorização a `@PreAuthorize` por método — e `PartnerController` não tem NENHUM `@PreAuthorize`. O único gate que resta é o `InternalRequestFilter`, que apenas confirma que a requisição chegou via gateway com o segredo interno e um `X-User-Id` presente — não valida `role/authority`. O gateway, por sua vez, só valida a assinatura do JWT e injeta os headers; não há restrição de `role` por rota para `/partner/**`. Curiosamente o gateway TEM um mecanismo pronto para esse tipo de restrição — `verifyPartnerURL()` em `SecurityFilter.java` (~linha 126) já restringe JWTs com `loginType=="PARTNER"` a um allowlist de prefixos (`/partner/api/v1/partners/me`, `/cnpi`, `/billing/partner/`) — mas essa restrição não existe para nenhum OUTRO `loginType` (ex.: usuário comum de um tenant) alcançando as mesmas rotas administrativas. No frontend admin, a rota `/parceiros/contadores` é protegida só pelo `authGuard` (checa apenas se existe um token em `sessionStorage`, não o papel do usuário). Resultado: qualquer usuário autenticado no admin (ou qualquer portador de um JWT válido de `loginType` diferente de `PARTNER`) pode listar todos os parceiros da plataforma, ver/editar qualquer parceiro por ID, ver dados de engajamento/indicação de qualquer tenant por ID, e aprovar/reprovar/inativar qualquer parceiro.

Condições de exploração: Sem pré-condição de configuração — comportamento padrão do código em qualquer ambiente. Único requisito: um JWT válido de qualquer usuário autenticado (gateway apenas valida assinatura/expiração, não checa `role` para esta rota). Endpoints `/me/*` (dashboard, convites, payout-info, comissões) são seguros — derivam `partnerId` do JWT via `SecurityUtils.getPartnerId()`, não de path param. POST `/api/v1/partners` e GET `/cnpi/{cnpi}` são intencionalmente públicos (solicitação de parceria e consulta de CNPJ) e não fazem parte deste achado.

Baixa

F2 — Redis: default hardcoded "test-password" quando REDIS_PASSWORD não está setada**Categoria:** Chaves expostas

compose.yaml	27
compose.yaml	33
billing-service/src/main/resources/application.yaml	20

```
command: ["redis-server", "--appendonly", "yes", "--requirepass", "${REDIS_PASSWORD:-test-password}"]
test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD:-test-password}", "ping"]

# billing-service/application.yaml
password: ${REDIS_PASSWORD:-test-password}
```

Por que é explorável: Se a variável de ambiente REDIS_PASSWORD não estiver setada (esquecimento de deploy, .env incompleto), tanto o Redis quanto o billing-service sobem silenciosamente com a senha fixa "test-password" em vez de falhar no startup — um valor previsível, presente em texto claro no repositório. Diferente de JWT_SECRET/DB_PASS/ASAAS_*, que usam apenas \${VAR} sem fallback.

Condições de exploração: Requer: (a) REDIS_PASSWORD ausente no ambiente e (b) a porta do Redis alcançável pela rede (não apenas localhost). Em docker-compose local isolado o risco é baixo; em um deploy onde a rede do Redis for exposta sem essa env var, um atacante ganha acesso de escrita ao Redis usado pelo DistributedLock dos jobs do billing-service.

Média

F3 — auth-service: HTML de e-mail transacional monta corpo com dado de usuário sem escaping**Categoria:** Inputs sem tratamento (XSS/HTML injection)

auth-service/src/main/java/com/erp/authservice/services/EmailConsumerService.java	104
auth-service/src/main/java/com/erp/authservice/services/EmailConsumerService.java	151
auth-service/src/main/java/com/erp/authservice/services/EmailConsumerService.java	183-184
auth-service/src/main/java/com/erp/authservice/services/EmailConsumerService.java	258
auth-service/src/main/java/com/erp/authservice/services/EmailConsumerService.java	301, 304, 380, 383

```
// linha 104
"<h2 style='color: #0056b3;'>Olá, %s!</h2>".formatted(name)

// linha 258 – texto livre digitado pelo parceiro, direto no HTML
"<blockquote>%s</blockquote>".formatted(message)
```

Por que é explorável: Todos os métodos send*Email montam o corpo HTML do e-mail via String.format/text block com %s, concatenando diretamente name, partnerName, clientName, clientCnpj e — o caso mais sensível — message (texto livre digitado pelo parceiro no follow-up de um convite, campo extra.get("message") do payload Kafka), sem nenhuma função de escaping (nenhum import de HtmlUtils/OWASP encoder no arquivo). Um parceiro mal-intencionado pode injetar HTML/markup arbitrário no corpo do e-mail enviado a clientes/tenants.

Condições de exploração: Requer que um parceiro cadastrado use o endpoint de follow-up (POST /me/convites/{referralId}/followup) com um campo message contendo markup HTML. A maioria dos clientes de e-mail bloqueia execução de script, mas HTML injection ainda permite phishing (links falsos, layout falsificado se passando por comunicação oficial) e quebra de layout — por isso média, não alta/crítica.

Baixa

F4 — gateway: JWT_SECRET sem validação de tamanho mínimo no startup (inconsistente com auth-service)

Categoria: Chaves expostas

gateway/src/main/java/com/erp/gateway/security/SecurityFilter.java	30-31
auth-service/src/main/java/com/erp/authservice/infra/TokenService.java	27-32

```
// gateway/SecurityFilter.java – sem @PostConstruct, sem checagem
@Value("${api.security.jwt.secret}")
private String secret;

// auth-service/TokenService.java – valida no startup
@PostConstruct
void validateSecret() {
    if (secret == null || secret.length() < 32) throw new IllegalStateException(...);
}
```

Por que é explorável: auth-service falha rápido no startup se JWT_SECRET tiver menos de 32 caracteres; o gateway, que também assina/valida o mesmo tipo de token, não tem a mesma guarda. Não é um valor hardcoded (ambos usam apenas \${VAR}, sem default) — é uma inconsistência de hardening: se algum dia os dois serviços receberem valores diferentes de JWT_SECRET (erro de config), o gateway aceitaria um secret fraco silenciosamente em vez de recusar subir.

Condições de exploração: Baixo impacto prático hoje, pois ambos os serviços leem a mesma env var obrigatória sem default — a falha do auth-service já impediria o sistema de funcionar. Vira relevante só se as duas envs divergirem (configuração incorreta por serviço).

Informativa

F5 — Grafana local com senha hardcoded no compose.yaml (achado pré-existente, já rastreado)

Categoria: Chaves expostas

compose.yaml	139
--------------	-----

```
GF_SECURITY_ADMIN_PASSWORD=admin123
```

Por que é explorável: Senha de admin do Grafana em texto claro no compose.yaml. Já identificado em auditoria anterior desta mesma base (scan de secrets vazados, 2026-08-11); reconfirmado presente. Escopo é infra de observabilidade local/dev, não credencial de dado de cliente.

Condições de exploração: Só explorável se a porta 3000 (Grafana) estiver exposta a uma rede não confiável — em docker-compose local isolado o risco é nulo.

Informativa

F6 — cadastro-service / partner-service: /actuator/loggers sem autenticação (achado pré-existente, já rastreado)

Categoria: Banco sem tranca (isolamento de tenant)

cadastro-service/src/main/java/com/erp/cadastroservice/infra/filter/InternalRequestFilter.java	public-exempt paths
partner-service/src/main/java/com/erp/partnerservice/infra/filter/InternalRequestFilter.java	72-73

```
if (path.startsWith("/actuator/loggers")) return true; // isento de auth
```

Por que é explorável: /actuator/loggers permite ler e alterar o nível de log em runtime sem autenticação, isento deliberadamente para um painel de diagnóstico interno. Já mapeado em auditoria anterior (F2 do projeto, severidade LOW) — não é roteado pelo gateway, só acessível se a porta do serviço for alcançada diretamente.

Condições de exploração: Requer acesso direto à porta do serviço (8086/8087), não exposta via gateway/rotas públicas. Risco interno, já registrado como item de backlog de segurança.

Tabela consolidada de achados

Severidade	Arquivo:linha	Descrição
Crítica	PartnerController.java:1-247 (+5)	F1 — partner-service: PartnerController sem nenhuma autorização — CRUD administrativo aberto a qualquer usuário autenticado
Baixa	compose.yaml:27 (+2)	F2 — Redis: default hardcoded "test-password" quando REDIS_PASSWORD não está setada
Média	EmailConsumerService.java:104 (+4)	F3 — auth-service: HTML de e-mail transacional monta corpo com dado de usuário sem escaping
Baixa	SecurityFilter.java:30-31 (+1)	F4 — gateway: JWT_SECRET sem validação de tamanho mínimo no startup (Inconsistente com auth-service)
Informativa	compose.yaml:139	F5 — Grafana local com senha hardcoded no compose.yaml (achado pré-existente, já rastreado)
Informativa	InternalRequestFilter.java: public-exempt paths (+1)	F6 — cadastro-service / partner-service: /actuator/loggers sem autenticação (achado pré-existente, já rastreado)

Recomendações priorizadas

Crítica

P1. Adicionar `@PreAuthorize` em todos os métodos administrativos de `PartnerController` (`findAll`, `findById`, `listIndicacoes`, `origemPorTenant`, `engajamentoPorTenant`, `update`, `approve`, `reject`, `inactivate`), restritos a um authority equivalente a `PARTNER_MANAGE` — seguindo o padrão já usado em `SubscriptionController/RoleController`. Referência: F1.

Média

P2. Adicionar guard de tipo/role no route Angular `/parceiros/**` do `erp-front-end-admin` (hoje só checa presença de token) para refletir a restrição de backend assim que ela existir — defesa em profundidade, não substitui P1.

Média

P3. Sanitizar (ex.: `org.springframework.web.util.HtmlUtils.htmlEscape`) todo dado de usuário concatenado no HTML dos e-mails em `EmailConsumerService`, com atenção prioritária ao campo `message` livre do `follow-up`. Referência: F3.

Baixa

P4. Remover o default `REDIS_PASSWORD:-test-password` de `compose.yaml` e `billing-service/application.yaml` — falhar no startup se a env var não estiver setada, no mesmo padrão de `JWT_SECRET/DB_PASS`. Referência: F2.

Baixa

P5. Adicionar `@PostConstruct` de validação de tamanho mínimo do `JWT_SECRET` no gateway (mesma guarda que já existe em `TokenService` do `auth-service`). Referência: F4.

Informativa

P6. Girar a senha do Grafana em `compose.yaml` para uma env var sem default, e revisitar o item de backlog já existente sobre `/actuator/loggers` sem autenticação. Referências: F5, F6.

Issues para o GitHub

Texto completo pronto para colar em issues do GitHub. Cada bloco começa em **--- ISSUE n ---** e termina em **--- FIM ISSUE n ---**.

```
--- ISSUE 1 ---
# [Segurança] partner-service: PartnerController sem nenhuma autorização – CRUD administrativo aberto a qualquer usuário autenticado

**Labels sugeridas:** security, severity:critical

## Problema
partner-service não impõe autorização na cadeia de filtros do Spring Security (anyRequest().permitAll()), delegando 100% da autorização a @PreAuthorize por método – e PartnerController não tem NENHUM @PreAuthorize. O único gate que resta é o InternalRequestFilter, que apenas confirma que a requisição chegou via gateway com o segredo interno e um X-User-Id presente – não valida role/authority. O gateway, por sua vez, só valida a assinatura do JWT e injeta os headers; não há restrição de role por rota para /partner/**. Curiosamente o gateway TEM um mecanismo pronto para esse tipo de restrição – verifyPartnerURL() em SecurityFilter.java (~linha 126) já restringe JWTs com loginType=="PARTNER" a um allowlist de prefixos (/partner/api/v1/partners/me, /cnpj/, /billing/partner/) – mas essa restrição não existe para nenhum OUTRO loginType (ex.: usuário comum de um tenant) alcançando as mesmas rotas administrativas. No frontend admin, a rota /parceiros/contadores é protegida só pelo authGuard (checa apenas se existe um token em sessionStorage, não o papel do usuário). Resultado: qualquer usuário autenticado no admin (ou qualquer portador de um JWT válido de loginType diferente de PARTNER) pode listar todos os parceiros da plataforma, ver/editar qualquer parceiro por ID, ver dados de engajamento/indicação de qualquer tenant por ID, e aprovar/reprovar/inativar qualquer parceiro.

## Evidência
- `partner-service/src/main/java/com/l/erp/partnerservice/api/controllers/PartnerController.java:1-247`
- `partner-service/src/main/java/com/l/erp/partnerservice/infra/config/SecurityConfig.java:15, 23-24`
- `gateway/src/main/resources/application.yml:13-18`
- `gateway/src/main/java/com/l/erp/gateway/security/SecurityFilter.java:~126 (verifyPartnerURL)`
- `Angular/erp-front-end-admin/src/app/pages/cadastros/parceiros/parceiros.service.ts:1-53`
- `Angular/erp-front-end-admin/src/app/util/authguard.ts:4-14`
...

// SecurityConfig.java
.authorizeHttpRequests(auth -> auth.anyRequest().permitAll())

// PartnerController.java – nenhum método tem @PreAuthorize
@GetMapping
public ResponseEntity<...> findAll(...) { ... }
@GetMapping("/{id}")
public ResponseEntity<...> findById(@PathVariable UUID id) { ... }
@PatchMapping("/{id}/approve")
public ResponseEntity<...> approve(@PathVariable UUID id, ...) { ... }
@GetMapping("/referrals/by-tenant/{tenantId}")
public ResponseEntity<...> origemPorTenant(@PathVariable Long tenantId) { ... }
...

## Condições de exploração
Sem pré-condição de configuração – comportamento padrão do código em qualquer ambiente. Único requisito: um JWT válido de qualquer usuário autenticado (gateway apenas valida assinatura/expiração, não checa role para esta rota). Endpoints /me/* (dashboard, convites, payout-info, comissões) são seguros – derivam partnerId do JWT via SecurityUtils.getPartnerId(), não de path param. POST /api/v1/partners e GET /cnpj/{cnpj} são intencionalmente públicos (solicitação de parceria e consulta de CNPJ) e não fazem parte deste achado.

## Impacto
Qualquer usuário autenticado (qualquer tenant, qualquer tipo de login exceto PARTNER) pode listar, ler, editar, aprovar, reprovar e inativar parceiros de toda a plataforma via a API pública do partner-service – exposição total de dados de parceiros/tenants e possibilidade de aprovar contas fraudulentas ou inativar parceiros legítimos.

## Sugestão de correção
```

Adicionar `@PreAuthorize` em cada método administrativo de `PartnerController` (findAll, findById, listIndicacoes, origemPorTenant, engajamentoPorTenant, update, approve, reject, inactivate) exigindo uma authority equivalente a `PARTNER\_MANAGE`, seguindo o padrão já usado em `SubscriptionController`/`RoleController`. Os endpoints `/me/\*` (já seguros, escopados por JWT) e `POST /api/v1/partners` + `GET /cnpj/{cnnpj}` (intencionalmente públicos) não precisam de mudança.

Critérios de aceite

- [] Todo método de `PartnerController` que recebe um `id`/`tenantId` arbitrário e não é `/me/\*` tem `@PreAuthorize` com uma authority de gestão de parceiros
- [] Teste de integração cobrindo 403 para usuário sem a authority em cada um desses endpoints
- [] Teste de integração cobrindo 200 para usuário com a authority
- [] `SecurityConfig.java` do partner-service documentado (comentário) explicando que a autorização é 100% via `@PreAuthorize`, não pelo filtro HTTP

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] auth-service: HTML de e-mail transacional monta corpo com dado de usuário sem escaping

Labels sugeridas: security, severity:medium

Problema

Todos os métodos sendEmail montam o corpo HTML do e-mail via String.format/text block com %, concatenando diretamente name, partnerName, clientName, clientCnpj e – o caso mais sensível – message (texto livre digitado pelo parceiro no follow-up de um convite, campo extra.get("message") do payload Kafka), sem nenhuma função de escaping (nenhum import de HtmlUtils/OWASP encoder no arquivo). Um parceiro mal-intencionado pode injetar HTML/markup arbitrário no corpo do e-mail enviado a clientes/tenants.

Evidência

```
- `auth-service/src/main/java/com/l/erp/authservice/services/EmailConsumerService.java:104`
- `auth-service/src/main/java/com/l/erp/authservice/services/EmailConsumerService.java:151`
- `auth-service/src/main/java/com/l/erp/authservice/services/EmailConsumerService.java:183-184`
- `auth-service/src/main/java/com/l/erp/authservice/services/EmailConsumerService.java:258`
- `auth-service/src/main/java/com/l/erp/authservice/services/EmailConsumerService.java:301, 304, 380, 383`
```

...

// linha 104

```
"<h2 style='color: #0056b3;'>Olá, %s!</h2>".formatted(name)
```

// linha 258 – texto livre digitado pelo parceiro, direto no HTML

```
"<blockquote>%s</blockquote>".formatted(message)
..."
```

Condições de exploração

Requer que um parceiro cadastrado use o endpoint de follow-up (POST /me/convites/{referralId}/followup) com um campo message contendo markup HTML. A maioria dos clientes de e-mail bloqueia a execução de script, mas HTML injection ainda permite phishing (links falsos, layout falsificado se passando por comunicação oficial) e quebra de layout – por isso média, não alta/crítica.

Impacto

HTML injection no corpo de e-mails transacionais enviados a tenants/clientes – vetor de phishing (links falsos, layout falsificado se passando por comunicação oficial da plataforma).

Sugestão de correção

Aplicar `org.springframework.util.HtmlUtils.htmlEscape(...)` (ou equivalente OWASP encoder) em todo dado de usuário (name, partnerName, clientName, clientCnpj, message, itens de features/gaps) antes de concatenar no HTML do e-mail em `EmailConsumerService`.

Critérios de aceite

- [] Todo `String.format`/text block que monta HTML de e-mail em `EmailConsumerService` escapa os parâmetros de entrada
- [] Teste unitário cobrindo que um `message` contendo `

```

--- ISSUE 3 ---
# [Segurança] Hardening de configuração: defaults perigosos e validação de secrets

**Labels sugeridas:** security, severity:low

## Problema
Três achados relacionados de hardening de configuração, agrupados por serem baixo impacto e mes-
ma categoria (chaves/secrets):

1. **Redis: default hardcoded "test-password" quando REDIS_PASSWORD não está setada** – Se a va-
riável de ambiente REDIS_PASSWORD não estiver setada (esquecimento de deploy, .env incompleto),
tanto o Redis quanto o billing-service sobem silenciosamente com a senha fixa "test-password"
em vez de falhar no startup – um valor previsível, presente em texto claro no repositório. Dife-
rente de JWT_SECRET/DB_PASS/ASAAS_*, que usam apenas ${VAR} sem fallback.

2. **gateway: JWT_SECRET sem validação de tamanho mínimo no startup (inconsistente com auth-ser-
vice)** – auth-service falha rápido no startup se JWT_SECRET tiver menos de 32 caracteres; o ga-
teway, que também assina/valida o mesmo tipo de token, não tem a mesma guarda. Não é um valor h-
ardcoded (ambos usam apenas ${VAR}, sem default) – é uma inconsistência de hardening: se algum-
dia os dois serviços receberem valores diferentes de JWT_SECRET (erro de config), o gateway ace-
itaria um secret fraco silenciosamente em vez de recusar subir.

3. **Grafana local com senha hardcoded no compose.yaml (achado pré-existente, já rastreado)** –
Senha de admin do Grafana em texto claro no compose.yaml. Já identificado em auditoria anterio-
r desta mesma base (scan de secrets vazados, 2026-08-11); reconfirmado presente. Escopo é infra
de observabilidade local/dev, não credencial de dado de cliente.

## Evidência
### 1. Redis default
- `compose.yaml:27`
- `compose.yaml:33`
- `billing-service/src/main/resources/application.yaml:20`
...
command: ["redis-server", "--appendonly", "yes", "--requirepass", "${REDIS_PASSWORD:-test-passw-
ord}"]
test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD:-test-password}", "ping"]

# billing-service/application.yaml
password: ${REDIS_PASSWORD:-test-password}
...

### 2. Gateway sem validação de JWT_SECRET
- `gateway/src/main/java/com/l/erp/gateway/security/SecurityFilter.java:30-31`
- `auth-service/src/main/java/com/l/erp/authservice/infra/TokenService.java:27-32`
...

// gateway/SecurityFilter.java – sem @PostConstruct, sem checagem
@Value("${api.security.jwt.secret}")
private String secret;

// auth-service/TokenService.java – valida no startup
@PostConstruct
void validateSecret() {
if (secret == null || secret.length() < 32) throw new IllegalStateException(...);
}
...

### 3. Grafana hardcoded
- `compose.yaml:139`
...
GF_SECURITY_ADMIN_PASSWORD=admin123
...

## Impacto
Baixo isoladamente (todos exigem uma pré-condição de rede/config incorreta para serem explorado-
s), mas compõem uma superfície de "falha silenciosa" – o sistema sobe funcional mesmo com confi-
guração insegura, em vez de falhar rápido.

## Sugestão de correção
- Remover o fallback `:-test-password` do Redis em `compose.yaml` e `billing-service/applicatio-
n.yaml`
- Adicionar `@PostConstruct` de validação de tamanho mínimo do `JWT_SECRET` em `gateway/Securit-
yFilter.java`, espelhando `TokenService.java` do auth-service

```

```
- Mover a senha do Grafana em `compose.yaml` para uma env var sem default
```

```
## Critérios de aceite
```

- [] Redis e billing-service falham no startup se `REDIS\_PASSWORD` não estiver setada
- [] Gateway falha no startup se `JWT\_SECRET` tiver menos de 32 caracteres
- [] `GF\_SECURITY\_ADMIN\_PASSWORD` vem de env var sem default no `compose.yaml`

```
--- FIM ISSUE 3 ---
```